

Hate Speech Detection on Tweets

Using RNN, LSTM, and Word2Vec Embeddings

6CS012 Final Portfolio Project | Herald College Kathmandu | Part III — Language Task

Student ID: 2408881 | Name: Prashun Baral | April, 2026
GitHub: github.com/prashunbaral/AIML — Assessment/NLP.ipynb

Abstract

Objective. This project tackles binary classification of tweets as either hate speech (racist or sexist) or normal, using a labelled dataset of 31,962 training samples from Twitter. The 13.3:1 class imbalance makes minority-class recall the genuinely difficult part of the problem — not overall accuracy.

Methods. Three models are built: a Simple RNN with trainable embeddings, a stacked LSTM with trainable embeddings, and the same LSTM seeded with pretrained Google News Word2Vec vectors (300 dimensions). All use Adam and binary cross-entropy with manually set class weights to compensate for imbalance. EarlyStopping and ReduceLROnPlateau manage training dynamics. Threshold optimisation via precision-recall curves is applied post-training.

Key Findings. At the default 0.5 threshold, all three models achieve ~93–94% accuracy. The stacked LSTM leads on weighted F1 (0.9467); Word2Vec, despite pretrained vectors, comes last due to 60.3% vocabulary coverage — nearly 40% of training vocabulary had no match in Google News. Threshold tuning pushes accuracy to 95–96% with the main gain in hate-class precision.

Conclusion. Architecture choices and class weighting mattered more than embedding source. Vocabulary coverage and domain match are more important than the size of the pretrained corpus. BERTweet pretrained on Twitter text is the most promising upgrade path.

1. Introduction

Hate speech spreads faster than moderators can catch it. Twitter makes this worse: posts are short, a retweet is one tap, and content can reach thousands before anyone flags it. Keyword filters catch slurs but miss context entirely — a tweet that looks neutral on its own can function as a threat depending on who is targeted and what happened the day before. A model reading one tweet in isolation has none of that background.

Bag-of-words approaches lose word order and negations. RNNs fixed this by carrying a hidden state through the sequence, but gradients decay over long sequences — by the first token the signal has mostly vanished. LSTMs address this with three gates (input, forget, output) controlling what enters

memory, what is discarded, and what carries forward, keeping gradients healthy over much longer sequences. Transformers are stronger still but come with memory, compute, and interpretability costs that make LSTMs still relevant for constrained setups.

This project trains all three architectures on a labelled hate speech dataset and examines the trade-off between false positives (normal tweets flagged as hate) and false negatives (hate that slips through) — two errors with very different real-world costs in content moderation.

2. Dataset

Source: Racist-Sexist Tweet Dataset (Kaggle — nabinpyakurel). Training set: 31,962 tweets, each labelled 0 (normal) or 1 (hate speech). Test set: 17,197 tweets with no labels — used for the Gradio inference demo only.

Label	Class	Count	Proportion
0	Normal	29,720	93.0%
1	Hate Speech	2,242	7.0%
—	Total (train)	31,962	100%

Table 1: Dataset class distribution (13.3:1 imbalance)

Tweet length runs from 3 to 34 words; mean and median both sit at 13. MAX_LEN = 13 was set at the 95th percentile of training lengths to avoid wasteful padding. Word clouds for the two classes look nothing alike — the hate class clusters tightly around a small set of targeted terms, while normal tweets spread across news, sport, and daily life.

Preprocessing pipeline: contraction expansion (contractions library: 'can't' → 'cannot'), lowercasing, URL and mention removal, hashtag stripping (# symbol removed, word kept), digit removal, non-alphabetic character removal, whitespace normalisation, stopword removal with a sentiment-preserving exception list (words like 'not', 'never', 'hate', 'kill', 'against' are kept), and short token filtering (tokens of length 1 dropped).

3. Methodology

3.1 Tokenisation and Sequence Preparation

The Keras Tokenizer was fitted on training data only (VOCAB_SIZE = 20,000). The training corpus contained 34,176 unique tokens — nearly double the limit; the bottom half by frequency was cut, with rare words mapping to a single OOV token. Sequences are padded or truncated to MAX_LEN = 13 tokens (post-padding, post-truncation). The 80/20 stratified train/test split preserves the 13.3:1 class ratio: 25,569 training samples and 6,393 test samples.

3.2 Model Architectures

Model 1 — Simple RNN: Embedding(20,000 → 128 dims), SimpleRNN(64 units), Dropout(0.3), Dense(32, ReLU), Dropout(0.2), sigmoid output. Plain RNNs suffer gradient decay over long

sequences, but at 13 tokens the decay does not fully bite.

Model 2 — Stacked LSTM: Embedding(20,000 → 128 dims), LSTM(64, return_sequences=True), Dropout(0.3), LSTM(32), Dropout(0.3), Dense(32, ReLU), sigmoid. Two LSTM layers in sequence — the second processes output from the first, working on an already-abstracted representation. Three gates control memory at each step, keeping gradients healthy.

Model 3 — LSTM + Word2Vec: Same LSTM stack but with 300-dimensional Google News Word2Vec vectors loaded into the embedding layer (weights=[embedding_matrix], trainable=True). Vocabulary matching: 12,050 hits out of 20,000 (60.3%). The 7,950 unmatched words — mostly Twitter-native slang, hashtag concatenations, deliberate misspellings — started at zero.

3.3 Training Configuration

Model	Class 0 Weight	Class 1 Weight	Learning Rate
Simple RNN	1.0	5.0	1e-3
Stacked LSTM	1.0	6.0	1e-3
LSTM + Word2Vec	1.0	5.0	3e-4

Table 2: Class weights and learning rates per model

Word2Vec gets a lower LR (3e-4) to avoid flattening the geometric structure it already encodes. EarlyStopping (patience=3, restore_best_weights=True) halts training when val_loss plateaus. ReduceLROnPlateau halves the rate after 2 flat epochs (min_lr=1e-6). Max 20 epochs, batch size 64.

4. Experiments and Results

4.1 RNN vs. LSTM Performance

At the default 0.5 threshold, all three models land in a narrow accuracy band. The LSTM edges the RNN on weighted F1 by a small margin. The gap is small because at 13 tokens there is not enough sequence length for vanishing gradients to severely damage the RNN.

Model	Accuracy	Precision	Recall	F1 (weighted)	Epochs
Simple RNN	94.48%	94.73%	94.48%	0.9459	5
Stacked LSTM	94.40%	95.06%	94.40%	0.9467	~6
LSTM + Word2Vec	93.27%	94.76%	93.27%	0.9384	~5

Table 3: Model performance at default 0.5 threshold

Per-class breakdown is more revealing: the RNN has precision 0.60 / recall 0.65 on hate speech; the LSTM has precision 0.58 / recall 0.72. The LSTM catches more hate speech but mislabels more normal tweets as hateful. For content moderation that is the right trade-off — a missed hate tweet does more damage than a false positive a reviewer can dismiss in three seconds.

4.2 Computational Efficiency

Model	Training Time	Epochs Run	Hardware
Simple RNN	~18.5s	5 (early stopped)	Colab T4 GPU
Stacked LSTM	~25.9s	~6	Colab T4 GPU
LSTM + Word2Vec	~16.6s	~5 (early stopped)	Colab T4 GPU

Table 4: Training time and hardware

All three ran under 30 seconds. Short sequences and early stopping kept training fast. The Simple RNN is faster per epoch but stopped at epoch 5, suggesting it hit its capacity ceiling quickly. Word2Vec was fastest overall — the lower LR converged quickly, though not to a better solution.

4.3 Training with Different Embeddings

Word2Vec was expected to outperform randomly initialised embeddings. In practice it came third. The core reason is vocabulary coverage: 60.3% of training vocabulary matched Google News, leaving 7,950 words zeroed out from the start. These are almost entirely Twitter-native — hashtag concatenations, slang, deliberate misspellings — exactly what the model needs to detect hate speech. A fully trainable embedding would have learned representations for those tokens from scratch; with Word2Vec, the model just saw nothing for nearly 40% of its vocabulary.

Domain mismatch compounds this. Google News is formal prose with consistent grammar. Twitter is structurally different. Even with trainable=True allowing the vectors to drift during training, the combination of low coverage and domain mismatch offset the benefit of the pretrained starting point. Domain-adapted embeddings (e.g., GloVe trained on Twitter or FastText with subword handling) would likely close this gap.

4.4 Model Evaluation — Threshold Optimisation

Precision-recall curves on a held-out validation split (15% of training data) found model-specific optimal thresholds:

Model	Optimal Threshold	Accuracy (tuned)	F1 Hate Class	F1 (weighted)
Simple RNN	0.849	96%	0.65	0.95
Stacked LSTM	0.812	96%	0.68	0.96
LSTM + Word2Vec	0.756	95%	0.63	0.95

Table 5: Threshold optimisation results

All three require a much higher threshold than 0.5. The RNN only fires a hate label at 84.9% confidence. This reflects the imbalance — the models learned to be conservative. Threshold choice is ultimately a business decision: a platform wanting zero missed hate tolerates more false positives; one protecting against over-moderation shifts the threshold up.

4.5 Error Analysis

Error analysis on the best model (LSTM + Word2Vec at default threshold) found 430 misclassifications out of 6,393 test samples (6.7%):

Category	Count	Rate
True Positives (hate correctly detected)	335	—
True Negatives (normal correctly classified)	5,628	—
False Positives (normal flagged as hate)	317	5.3%
False Negatives (hate missed as normal)	113	25.2% of hate class

Table 6: Confusion matrix breakdown for best model

False positives often involve tweets mentioning race, gender, or politically charged topics in a non-hateful context. The model fires on surface-level vocabulary rather than intent — a fundamental limitation of sequential models without attention. Three illustrative error types: (1) a tweet discussing anti-racism policy flagged because it contains target-group vocabulary; (2) a satirical tweet misclassified because the model cannot detect tone; (3) a quoted excerpt of hate speech — included to document it — misclassified because the surrounding context is invisible to the model.

Potential improvements: attention mechanisms to weight relevant tokens; longer MAX_LEN to avoid truncating context; domain-specific embeddings (GloVe-Twitter or BERTweet); data augmentation via back-translation on the minority class.

5. Conclusion and Future Work

5.1 Summary of Findings

The three models performed more similarly than expected. LSTM edged RNN on weighted F1 and minority-class recall. Word2Vec underperformed despite pretrained knowledge because 40% of task vocabulary was missing from Google News. Architecture choices and class weighting mattered more than embedding source. Threshold tuning raised accuracy 1–2 percentage points and improved hate-class precision substantially.

5.2 Limitations

Severe class imbalance (7% hate) makes overall accuracy a misleading metric — macro F1 and per-class recall are more informative. MAX_LEN = 13 truncates context in longer tweets. 60.3% Word2Vec coverage left nearly 8,000 words zeroed from the start. Context-free classification is a hard constraint — whether a word is hateful depends on who says it and in what context, information a single-tweet sequential model cannot access.

5.3 Future Work

Retraining embeddings on a large Twitter corpus, or switching to FastText (which handles subword information and is more robust to misspellings), would close the coverage gap directly. Back-translation or synonym substitution on minority-class tweets would diversify the training signal without duplicating identical examples. BERTweet — a BERT variant pretrained on Twitter text — is the most promising architecture upgrade: contextual bidirectional representations would capture the intent-level information that sequential models miss. An attention layer on top of the LSTM would also improve interpretability. Ensembling the three model predictions via majority vote is a low-cost improvement that tends to smooth out individual model failures on edge cases.